INDUSTRIAL TRAINING REPORT

HOSPITAL AI ASSISTANT

Submitted in partial fulfilment of the

Requirements for the award of

Degree of Bachelor of Technology in Data Science

Session: 2026-27

Submitted By

Name: Mayuri Agrawal

University Roll No.: 23ESKCX065 Semester/Branch: 4th Year (Data Science)

SUBMITTED TO:

Department of Computer Science & Engineering

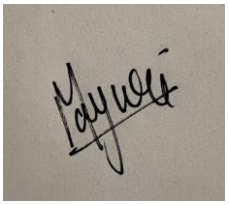
SWAMI KESHVANAND INSTITUTE OF TECHNOLOGY,
MANAGEMENT & GRAMOTHAN, JAIPUR

DECLARATION

I hereby declare that the Industrial Training Report entitled "Hospital AI Assistant" is an authentic record of the work carried out during my Industrial Training from 25 May 2026 to 25 July 2026 for the award of the degree of Bachelor of Technology in Data Science at Swami Keshavanand Institute of Technology, Management & Gramothan, Jaipur.

The project presented in this report is based on the implementation developed during the training period. The project integrates a Streamlit frontend, an SQLite database built from a healthcare dataset of 55,500 patient records, Google Gemini, prompt-based natural-languageto-SQL generation, environment-based configuration and modular test scripts.

I further declare that this report has been prepared for academic purposes and represents my understanding and implementation of the project.



Mayuri Agrawal
23ESKCX065

Date: 15-10-2026

Certified that the above statement made by the student is correct to the best of our knowledge and belief

Examined by:

Mr. Shivank Lawaniya

Head of Department

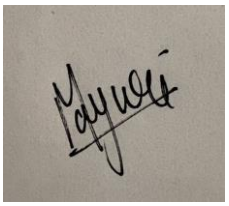
CERTIFICATE



ACKNOWLEDGEMENT

First and foremost, I wish to express my sincere thanks and gratitude to my esteemed Mentor **Mr. Sharthak Acharjee** who contributed to the successful completion of my Industrial Training through thoughtful reviews and valuable guidance.

Next, I would like to tender my sincere thanks to **Dr. Mehul Mahrishi** (Head of Computer Science & Engineering) for his co-operation and encouragement. I am thankful to **Mr. Shivank Lawaniya**, Coordinator (Industrial Training 2026-27) for his support, cooperation and motivation provided during the internship for constant inspiration, presence and blessings.



Mayuri Agrawal
23ESKCX065

About The Company

Celebal Technologies is a Jaipur-headquartered technology company that focuses on Data Science, Big Data, Data Engineering, Artificial Intelligence, Machine Learning and enterprise cloud solutions. The company helps organizations modernize their data platforms, integrate enterprise systems and use data-driven insights to improve business operations.

Its services include data engineering and analytics, ETL/ELT modernization, Generative AI and machine learning, cloud infrastructure and migration, enterprise integration, application modernization, and business intelligence. Celebal Technologies works with organizations across multiple industries and collaborates with major technology partners to deliver data and AI solutions. These areas make the company relevant to practical learning in Data Science, cloud technologies and modern data platforms.

Table of Contents

1. Introduction 8
    1.1 Key Benefits 8
    1.2 Project Overview 8
    1.3 Project Scope 9
    1.4 Intended Users 10
    1.5 Tools and Technologies 10
2. Tools and Technologies 11
    2.1 Python Programming Language 11
    2.2 Streamlit 11
    2.3 SQLite Database 12
    2.4 Pandas 13
    2.5 Google Gemini 13
    2.6 Prompt Engineering and Text-to-SQL 13
    2.7 Healthcare Dataset 13
    2.8 Environment Configuration (python-dotenv) 14
3. Dependencies 14
    3.1 Assumptions and Dependencies 14
    3.2 Application Dependencies 15
    3.3 AI Dependencies 15
    3.4 Database Dependencies 15
    3.5 Hardware Requirements 15
    3.6 Software Requirements 16
    3.7 Environment Configuration 16
4. Snapshots 18
    4.1 Login Page 18
    4.2 Dashboard Overview 18
    4.3 Patient Database 19
    4.4 Contact Us Page 19
    4.5 Hospital AI Assistant 20
5. Implementation and Testing 21
    5.1 System Architecture 21
    5.2 Dataset and Database Creation 21
    5.3 Database Access Layer 22
    5.4 Schema Reader 22
    5.5 Prompt Construction 22
    5.6 SQL Generation 23
    5.7 AI Question Answering 23
    5.8 User Interface 23
    5.9 Gemini Client Configuration 23
    5.10 Data Quality Observations 24
    5.11 Testing Strategy 24
    5.12 Database Testing 24
    5.13 Schema and Prompt Testing 24
    5.14 Gemini Connection Testing 24
    5.15 SQL Generation Testing 25
    5.16 Error Handling 25
    5.17 Security Measures 25

6. Result, Conclusion and Future Scope..... 25
6.1 Result..... 25
6.2 Conclusion 26
6.3 Future Scope..... 26

7.
References.....28

List of Figures

1. Login Page.....18
2. Dashboard Overview.....18
3. Patient Database.....19
4. Contact Us Page.....19
5. Hospital AI Assistant.....20

INTRODUCTION

1.1 Key Benefits:

The Hospital AI Assistant provides several important benefits:

1. Natural-Language Access to Patient Data

Users can ask questions about hospital records in plain English instead of writing SQL queries. The system converts the question into a database query with the help of a generative AI model.

2. Structured Data Retrieval

Patient information is stored in a relational database rather than in unstructured text. This allows precise operations such as filtering by age, condition or admission type, counting records and sorting results.

3. Schema-Aware SQL Generation

The database schema is read directly from the SQLite database and supplied to the AI model. As a result, the generated queries use the exact table and column names that exist in the database.

4. Safety-Oriented Prompting

The prompt instructs the model to generate only SELECT queries and explicitly forbids DELETE, UPDATE, INSERT, DROP, ALTER and CREATE statements, so that the stored data is not modified by generated queries.

5. General AI Question Answering

In addition to database queries, the application provides a question box through which users can send free-form questions to Google Gemini and read the response on the same page.

6. Lightweight and Portable

The project uses SQLite, which stores the entire database in a single file and does not require a separate database server. The application can therefore be run on an ordinary computer.

7. Modular Architecture

Database access, schema reading, prompt construction, AI agents and the user interface are placed in separate modules. This makes the project easier to test, maintain and extend.

8. Secure Configuration

The Google API key is loaded from an environment file and is not hard-coded in the source code.

1.2 Project Overview:

The Hospital AI Assistant is an AI-powered hospital data exploration and question-answering application developed using Python, Streamlit, SQLite and Google Gemini. It allows a user to browse patient records and to ask questions about hospital data in natural language.

The system is built around a healthcare dataset containing 55,500 patient admission records with 15 attributes, including patient age, gender, blood type, medical condition, doctor, hospital, insurance provider, billing amount, admission type, medication and test results. The dataset is loaded into an SQLite database so that it can be queried efficiently. The workflow designed for the natural-language query feature can be summarized as:

User → Natural-Language Question → Read Database Schema → Build Prompt → Gemini → SQL Query → Clean SQL → Execute on SQLite → Display Results

When a question is received, the schema reader obtains the names of the table columns from the database. The prompt builder combines the schema, a set of strict rules and the user's question into a single prompt. Gemini returns an SQL query, and the SQL agent removes any markdown formatting from the response. The cleaned query can then be executed on the SQLite database, and the result is returned as a pandas DataFrame that Streamlit can display as a table.

1.2.1 Current Implementation Status:

The Streamlit interface currently provides two functions: (a) a patient database viewer that displays the first 20 records of the patients table, and (b) a question box that sends the user's question to Gemini through the AI agent and displays the generated answer. The text-to-SQL components (schema reader, prompt builder, SQL generator and query executor) are implemented as independent modules and have been verified through dedicated test scripts. An orchestrator module, which will connect these components to the interface and route each question to the correct agent, has been created as a placeholder for the next stage of development (see Chapter 6).

1.3 Project Scope:

The project covers the development of an AI-powered hospital data assistant with the following functionalities:

- Conversion of a CSV healthcare dataset into an SQLite database
- Database connection and query-execution module
- Patient records viewer in the Streamlit interface
- Database schema reader
- Prompt builder for SQL generation
- Gemini-based natural-language-to-SQL generation
- Cleaning of AI-generated SQL output
- General AI question-answering interface
- Environment-based API key configuration
- Test scripts for the database, schema, prompt, Gemini connection and SQL generation The current implementation works with a single patients table. User authentication, document (PDF) upload and vector-based retrieval, voice interaction, image processing and automatic execution of AI-generated SQL from the interface are not part of the current version and are considered future enhancements. The application is

intended for learning and demonstration purposes and is not a clinical decision-support tool.

1.4 Intended Users:

The system can be used by different categories of users.

1. Hospital Administrative Staff

Administrative staff can browse patient admission records and, in the future, ask questions such as the number of emergency admissions or the total billing for a particular insurance provider.

2. Doctors and Clinical Staff

Clinical staff can use the application to look up records and obtain summaries of patient groups without writing database queries.

3. Students

Computer science and medical informatics students can use the project to understand how generative AI can be combined with relational databases.

4. Data Analysts and Researchers

Analysts can explore the healthcare dataset in natural language to identify trends in conditions, medications, admission types and test results.

5. Developers

Developers can use the modular code base as a starting point for building text-to-SQL and AIassistant applications.

1.5 Tools and Technologies:

Technology	Purpose
Python	Main programming language
Streamlit	Frontend web interface
SQLite	Database for patient records
Pandas	Loading CSV data and returning query results as DataFrames
Google Gemini	Generative AI model (gemini-flash-latest)
google-generativeai	Python SDK used by the application to call Gemini
google-genai	Newer Google GenAI SDK used in the connection and modellisting tests
python-dotenv	Loads the API key from the environment file
re (regular expressions)	Removes markdown formatting from generated SQL
sqlite3 / os	Python standard-library modules for database access and file paths

TOOLS AND TECHNOLOGIES

The development of the Hospital AI Assistant required the integration of technologies from the fields of web development, databases, data processing and generative artificial intelligence. The application is designed so that the frontend provides an interactive environment for the user, while separate Python modules manage database access, schema reading, prompt construction and communication with the generative AI service.

The major technologies used in the project include Python, Streamlit, SQLite, Pandas and Google Gemini. Additional libraries are used for environment configuration and text processing.

2.1 Python Programming Language:

Python is the primary programming language used throughout the project. It provides the foundation for the user interface, database operations, data processing, AI integration and testing.

Python was selected because it has a large ecosystem of libraries for data handling and artificial intelligence. The project requires operations such as loading a CSV file, creating a database, running SQL queries, building prompts, communicating with Gemini and displaying results in a web page. All of these tasks can be implemented efficiently in Python.

2.1.1 Key Features of Python Used:

The major Python capabilities used in the project include:

- Modular programming (packages for database, agents, prompts and utils)
- File and path handling
- Database connectivity through sqlite3
- Data processing with Pandas
- String processing and regular expressions
- Environment-variable handling
- AI model integration
- Script-based testing

2.1.2 Role in the Project:

Python is used in every layer of the application. The Streamlit interface is written in Python, the database scripts use Python's sqlite3 module and Pandas, and the AI agents use the Gemini SDK from Python. The overall role of Python can be represented as:

Python → Frontend + Database Access + Prompt Building + AI Agents + Testing

2.2 Streamlit:

Streamlit is used to develop the interactive frontend of the Hospital AI Assistant. It allows web interfaces to be created entirely in Python without a separate HTML, CSS or JavaScript

framework. This was suitable for the project because the interface mainly consists of buttons, a text box and tables.

2.2.1 Features of Streamlit: Streamlit provides:

1. Rapid development of data applications
2. Easy integration with Python code
3. Interactive widgets such as buttons and text inputs
4. Built-in display of Pandas DataFrames as tables
5. Multi-column page layouts
6. Simple local deployment with a single command
7. Easy integration with AI applications

2.2.2 Frontend Workflow:

The frontend receives the user's input, calls the required Python modules and displays the response on the same page.

USER
↓
STREAMLIT INTERFACE
↓
PYTHON MODULES (DATABASE / AI AGENT)
↓
SQLITE DATABASE / GOOGLE GEMINI
↓
RESPONSE
↓ STREAMLIT INTERFACE

2.3 SQLite Database:

SQLite is used as the database of the application. It stores the patient records in a table named patients, which is created from the healthcare dataset by the database-creation script.

SQLite is suitable for this project because it is lightweight, requires no installation of a database server, stores the whole database in a single file (hospital.db) and fully supports standard SQL. Since the generated queries are executed on SQLite, the prompt explicitly asks Gemini to produce SQLite-compatible SQL.

2.3.1 Role in the Project:

- Stores 55,500 patient admission records
- Provides the schema that is supplied to the AI model
- Executes SELECT queries and returns the results
- Supplies the records displayed in the patient database viewer

2.4 Pandas:

Pandas is used for data handling. In the database-creation script, Pandas reads the CSV file into a DataFrame and writes it into the SQLite table using the to_sql function. In the database access module, the read_sql_query function executes an SQL query and returns the result as a DataFrame, which Streamlit displays directly as an interactive table.

2.5 Google Gemini:

Google Gemini is used as the generative AI layer of the system. The application accesses the model through Google's Generative AI SDK and is configured to use the gemini-flash-latest model, which is a fast model suitable for interactive applications.

2.5.1 Role in the Project: Gemini
is used for:

- Converting natural-language questions into SQL queries
- Answering general questions typed by the user in the question box

The API key required to access Gemini is read from an environment variable and is never written directly in the code.

2.6 Prompt Engineering and Text-to-SQL:

Text-to-SQL is the task of converting a question written in natural language into an SQL query. Large language models can perform this task well when they are given the database schema and clear instructions.

In this project the prompt contains the role of the model (an expert SQLite SQL generator), the database schema, seven rules and the user's question. The rules require valid SQLite SQL only, no explanation, no markdown, SELECT queries only, no data-modifying statements and use of the exact table and column names. This structured prompt reduces the chance of receiving unusable or unsafe output.

User Question
↓
Schema + Rules + Question = Prompt
↓
Gemini
↓ SQL
Query

2.7 Healthcare Dataset:

The data used by the project is stored in the file healthcare_dataset.csv. It contains 55,500 records and 15 columns. The admissions in the dataset range from May 2019 to May 2024, and the patient ages range from 13 to 89 years. The columns are described below.

No.	Column	Type	Description
1	Name	TEXT	Patient name

2	Age	INTEGER	Age of the patient in years
3	Gender	TEXT	Male or Female
No.	**Column**	**Type**	**Description**
4	Blood Type	TEXT	One of eight blood groups (A+, A-, B+, B-, AB+, AB-, O+, O-)
5	Medical Condition	TEXT	Arthritis, Asthma, Cancer, Diabetes, Hypertension or Obesity
6	Date of Admission	TEXT	Date on which the patient was admitted
7	Doctor	TEXT	Doctor responsible for the patient
8	Hospital	TEXT	Hospital in which the patient was admitted
9	Insurance Provider	TEXT	Aetna, Blue Cross, Cigna, Medicare or UnitedHealthcare
10	Billing Amount	REAL	Amount billed for the admission
11	Room Number	INTEGER	Room allotted to the patient
12	Admission Type	TEXT	Elective, Emergency or Urgent
13	Discharge Date	TEXT	Date on which the patient was discharged
14	Medication	TEXT	Aspirin, Ibuprofen, Lipitor, Paracetamol or Penicillin
15	Test Results	TEXT	Normal, Abnormal or Inconclusive

2.8 Environment Configuration (python-dotenv):

The python-dotenv library loads configuration values from a .env file into the environment when the application starts. The project uses it to read the GOOGLE_API_KEY value, so that sensitive credentials are kept outside the source code and can be changed without modifying the program.

DEPENDENCIES

The Hospital AI Assistant depends on several software packages, system resources, an external AI service and configuration values. Correct installation and configuration of these dependencies is necessary for the successful execution of the application.

The project is designed to run on common operating systems and uses a Python virtual environment to isolate project dependencies.

3.1 Assumptions and Dependencies:

The design and implementation of the project are based on a few key assumptions regarding the system environment, tools and resources. These assumptions serve as the foundation for the smooth operation of the project.

3.1.1 Assumptions:

The following assumptions are made for the application:

1. A stable internet connection is available for calling the Gemini service.
2. A valid Google API key is configured in the .env file.
3. Required Python packages are installed.
4. The SQLite database has been created from the CSV file before the application is started.
5. The scripts are executed from the project's root folder.
6. The patients table uses the column names given in Section 2.7.
7. Sufficient memory is available to load the records for display.
8. Required environment variables are correctly configured.

3.2 Application Dependencies:

The application layer uses the following packages:

Streamlit

Used for building the web interface.

Pandas

Used for loading the CSV file, writing the database table and reading query results.

python-dotenv

Used for loading configuration values from the environment file.

3.3 AI Dependencies:

The AI functionality requires:

google-generativeai

The SDK used by the application to configure the API key, create the Gemini model and generate content.

google-genai

The newer Google GenAI client used by the test scripts to check the connection and list the available models.

Google API key

A key obtained from Google AI Studio, required to authenticate every request to Gemini.

3.4 Database Dependencies:

SQLite is used for persistent storage. It is accessed through Python's built-in sqlite3 module, so no additional database software or driver needs to be installed. The database file is created in the database folder of the project.

3.5 Hardware Requirements:

3.5.1 Minimum Practical Configuration:

- Processor: Modern dual-core or equivalent
- RAM: 4 GB
- Storage: At least 2 GB free
- Stable internet connection
- Modern web browser

3.5.2 Recommended Configuration:

- Modern quad-core processor
- RAM: 8 GB
- SSD storage
- Stable broadband connection

The dataset contains 55,500 records and is small enough to be handled comfortably on an ordinary laptop, because the interface displays only a limited number of records at a time.

3.6 Software Requirements:

The project can run on:

- Windows
- Linux
- macOS

The recommended environment includes:

- Python 3
- Virtual environment
- pip
- Streamlit, Pandas, python-dotenv, google-generativeai and google-genai • Modern browser

After installing the packages, the database is created with the command python database/create_database.py and the application is started with the command streamlit run app.py from the project folder.

3.7 Environment Configuration:

The project uses environment variables for configuration. The important configuration values are:

- GOOGLE_API_KEY – the key used to access Gemini
- Database path – derived from the project folder (database/hospital.db)
- Data path – the location of healthcare_dataset.csv

Sensitive credentials should not be directly hard-coded into the source code, and the .env file should not be shared publicly.

Advantages

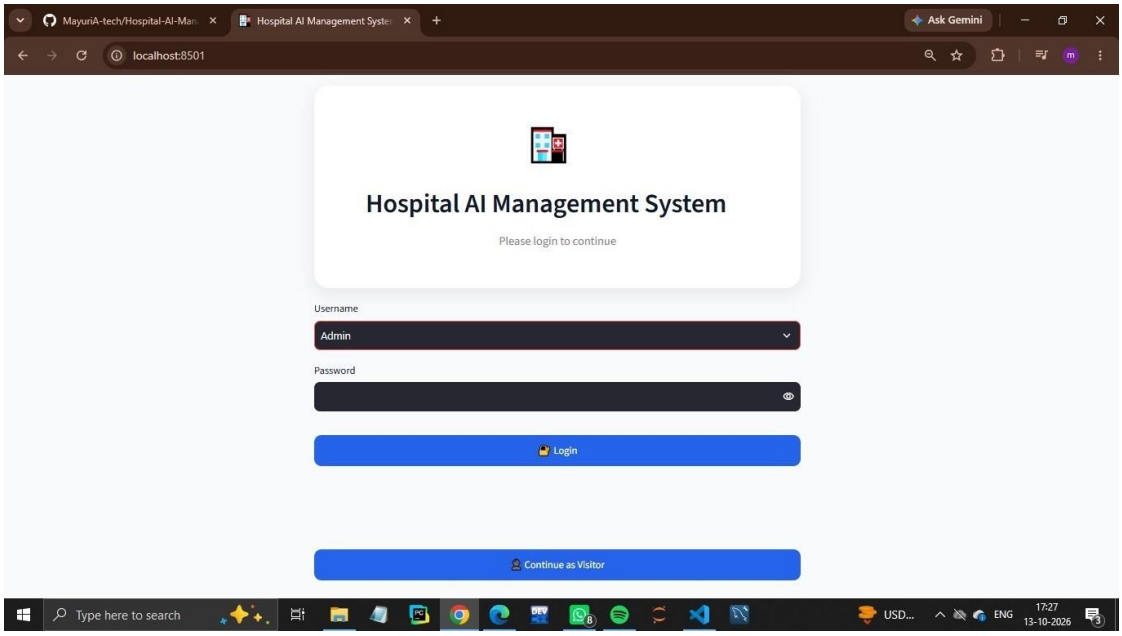
- Improved security

- Easy configuration
- Different settings for development and production
- Prevention of accidental credential exposure

SNAPSHOTS

This chapter presents the visual representation of the Hospital AI Assistant. The screenshots were captured from the running application and from the test scripts, and they demonstrate the working interface and the major components of the project.

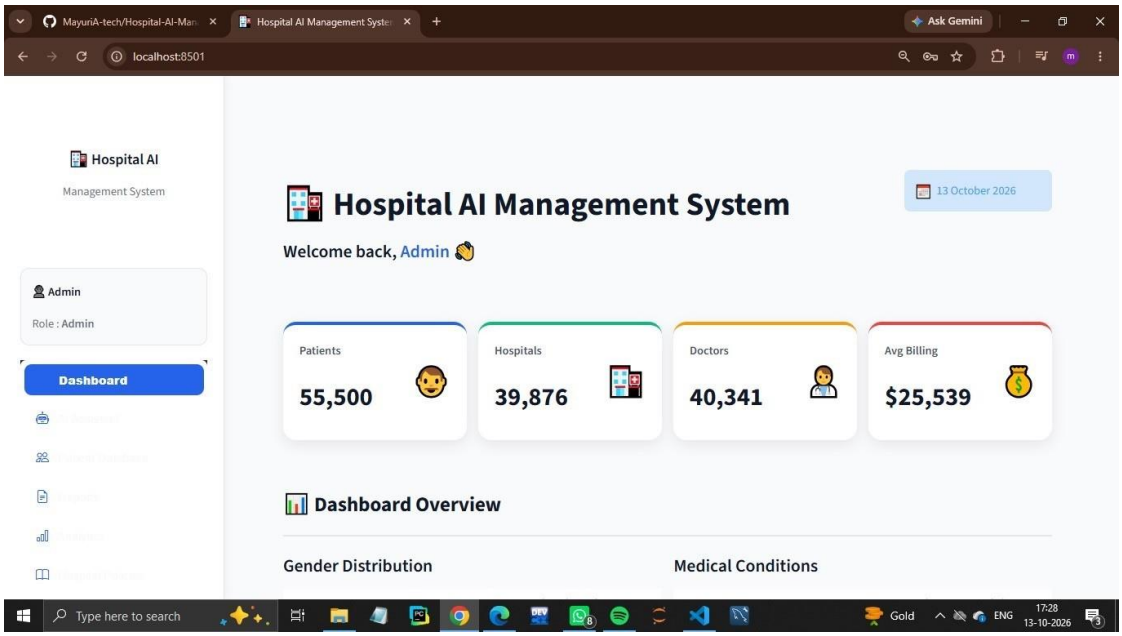
4.1 Login Page:



1. Login Page

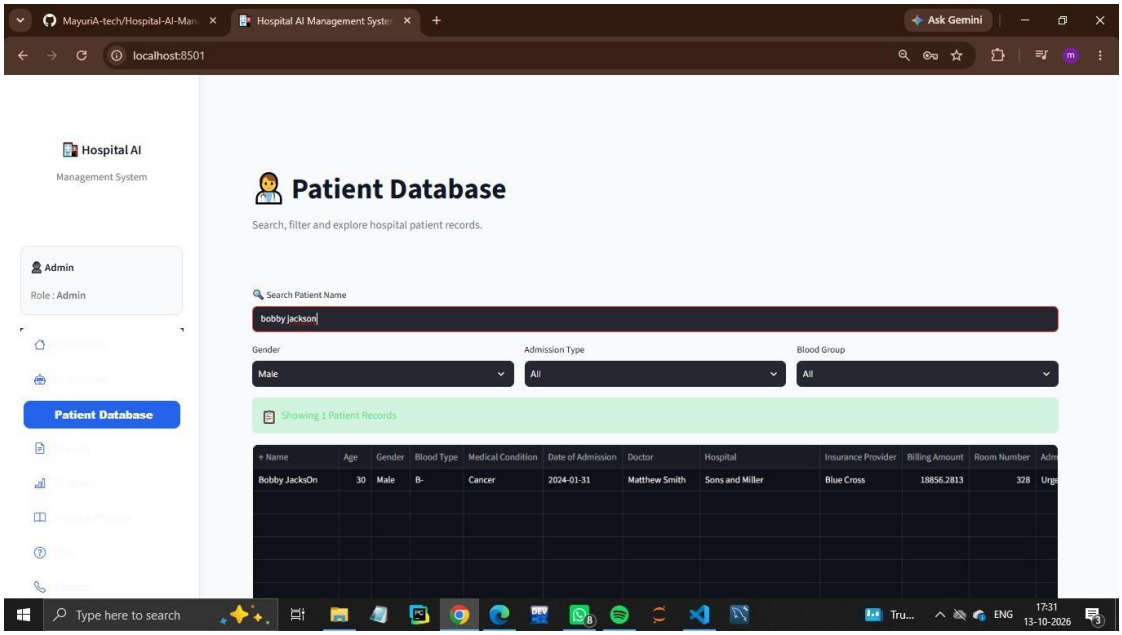
4.2 Dashboard

Overview:



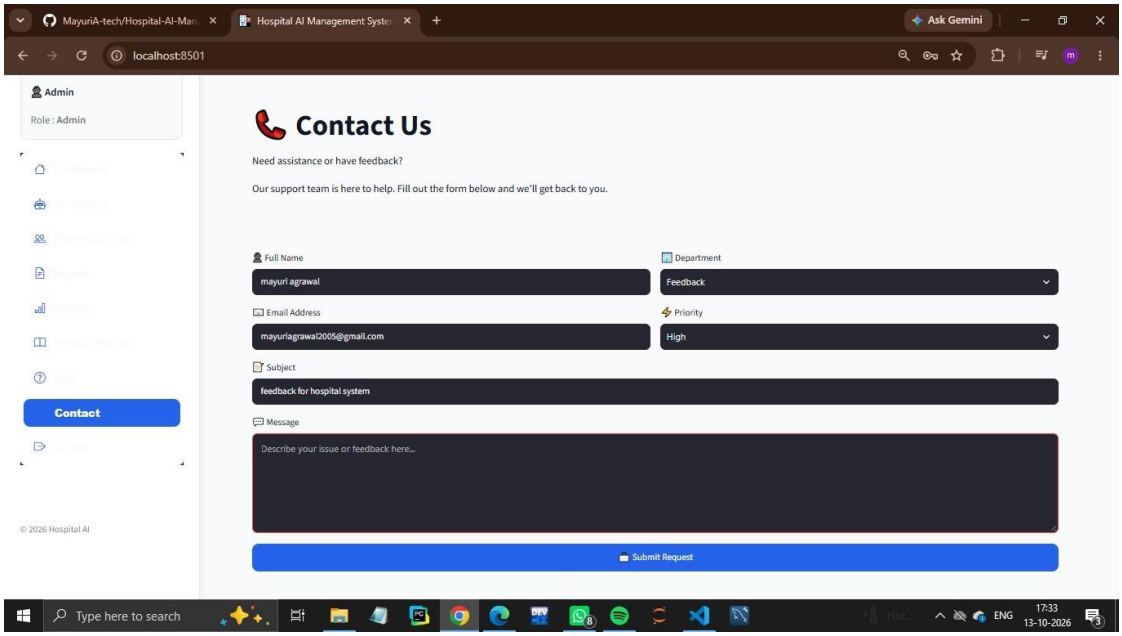
2. Dashboard Overview

4.3 Patient Database:



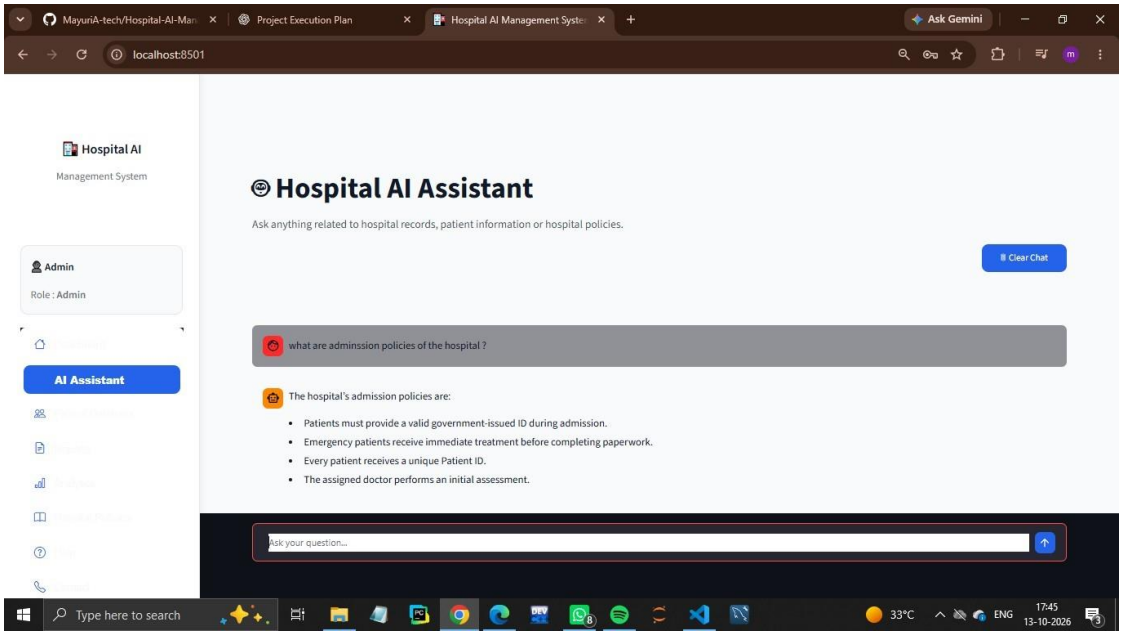
3. Patient Database

4.4 Contact Us Page:



4. Contact Us Page

4.5 Hospital AI Assistant:



5. Hospital AI Assistant

IMPLEMENTATION AND TESTING

Implementation involves converting the proposed design into a functional application. The Hospital AI Assistant was implemented as a modular system consisting of a frontend, a database layer, prompt-building and schema components, AI agents and a Gemini client.

The primary workflow is:

Load Dataset → Create Database → Read Schema → Build Prompt → Generate SQL → Clean SQL → Execute Query → Display Results.

5.1 System Architecture:

The system consists of several major layers.

- **Presentation Layer** – implemented using Streamlit (app.py).
- **Agent Layer** – the SQL agent (sql_agent.py) and the AI agent (rag_agent.py); an orchestrator module is reserved for routing questions.
- **Prompt Layer** – the SQL prompt builder (sql_prompt.py).
- **Data Layer** – SQLite database, the database access module and the schema reader. • **AI Layer** – Google Gemini accessed through the Gemini client (gemini_client.py). The separation of these layers improves maintainability. The project folder is organized as follows:

```
Hospital-AI-Assistant/
├── app.py                    (Streamlit interface)
├── agents/
│   ├── sql_agent.py          (natural language → SQL)
│   ├── rag_agent.py          (general AI question answering)
│   └── orchestrator.py       (reserved for routing)
├── database/
│   ├── create_database.py    (CSV → SQLite)
│   ├── hospital_database.py  (connection and queries)
│   ├── test_database.py
│   └── hospital.db
├── data/healthcare_dataset.csv
├── prompts/sql_prompt.py
├── utils/
│   ├── gemini_client.py
│   └── schema_reader.py
└── test_*.py                 (test scripts)
```

5.2 Dataset and Database Creation:

The first step is to convert the CSV dataset into a database. The script create_database.py reads healthcare_dataset.csv into a Pandas DataFrame, connects to database/hospital.db and writes the data into a table named patients using the to_sql function. After the insertion, the script prints the total number of records inserted, which for the supplied dataset is 55,500. healthcare_dataset.csv

↓

Pandas DataFrame

↓

to_sql()

↓

SQLite table: patients

5.3 Database Access Layer:

The module hospital_database.py provides the functions used to communicate with the database.

- **get_connection()** creates and returns a connection to the SQLite database. The database path is calculated from the location of the file, so it works regardless of the current directory.
- **execute_query(query)** executes a SELECT query using Pandas and returns the result as a DataFrame. The connection is always closed in a finally block, even if the query fails.
- **get_patients(limit=20)** returns the first records of the patients table and is used by the patient database viewer.

5.4 Schema Reader:

The module schema_reader.py reads the structure of the patients table using the PRAGMA table_info command and converts it into a readable text description. The schema is generated from the live database, so if the table changes the prompt is updated automatically. The output has the form:

```
Table: patients
Columns:
- Name
- Age
- Gender
- Blood Type
- Medical Condition...
- Test Results
```

5.5 Prompt Construction:

The function build_sql_prompt in sql_prompt.py combines the schema and the question into the prompt that is sent to Gemini. The prompt contains the following rules:

1. Generate only valid SQLite SQL
2. Do not explain the query
3. Do not use markdown
4. Do not write ```sql
5. Generate only SELECT queries
6. Never generate DELETE, UPDATE, INSERT, DROP, ALTER or CREATE
7. Use the exact table and column names from the schema

System Role + Database Schema + Rules + User Question

↓

Prompt Sent to Gemini

5.6 SQL Generation:

The function generate_sql in sql_agent.py performs the complete text-to-SQL step. It reads the schema, builds the prompt, sends it to Gemini and returns the SQL query. Because language models sometimes wrap code in markdown fences, the helper function clean_sql uses regular expressions to remove the ```sql and ``` markers and trims extra whitespace, so that the returned text is a plain SQL statement.

User Question
↓
get_schema()
↓
build_sql_prompt()
↓
Gemini generate_content()
↓
clean_sql()
↓
SQL Query

For example, the question “Show all diabetic patients older than 60” is converted into a query that filters the Medical Condition column for Diabetes and the Age column for values above 60.

5.7 AI Question Answering:

The module rag_agent.py contains the function ask_ai, which sends a question to Gemini and returns the text of the response. It is used by the question box in the Streamlit interface. In the current version the question is sent directly to the model; connecting this agent with the database retrieval step, so that answers are grounded in the stored records, is part of the planned work.

5.8 User Interface:

The interface is implemented in app.py using Streamlit. It is configured as a wide page with the title “Hospital AI Assistant” and contains the following elements:

- **Patient Database button** – calls get_patients() and displays the Patient Records table.
- **AI Medical Assistant button** – reserved for the full assistant and currently shows a coming-soon message.
- **Ask a Question box and Submit button** – sends the typed question to ask_ai() and shows the AI Response.

The interface checks that the question is not empty before calling the model.

5.9 Gemini Client Configuration:

The module gemini_client.py loads the .env file, configures the Google Generative AI SDK with the GOOGLE_API_KEY value and creates a single Gemini model object (gemini-flashlatest).

The function get_gemini_model returns this object to the agents, so both the SQL agent and the AI agent share one configured client.

5.10 Data Quality Observations:

While working with the dataset, the following data-quality characteristics were observed:

- 534 records are exact duplicates of other records.
- 108 records contain a negative billing amount.
- Patient names have inconsistent capitalization (for example, mixed upper and lower case letters) and some contain prefixes such as “Mr.”, “Ms.” or “Mrs.”.
- Some hospital names contain stray words or trailing commas.
- Column names contain spaces (for example, “Medical Condition”), so they must be enclosed in quotation marks in SQL queries.

The data is stored as supplied so that the original dataset remains unchanged. Data cleaning is included in the future scope of the project.

5.11 Testing Strategy:

Testing was performed using small Python scripts that exercise each component separately, so that a failure can be traced to a specific module. The main testing categories were:

1. Database testing
2. Schema reader testing
3. Prompt testing
4. Gemini connection and model testing
5. SQL generation testing

5.12 Database Testing:

The script test_database.py connects to hospital.db and prints the column names of the patients table using PRAGMA table_info. It verifies that the database file exists, that the patients table was created from the CSV file and that all 15 columns are present with the expected names. Additional checks confirmed that the table contains 55,500 rows and that get_patients() returns the requested number of records.

5.13 Schema and Prompt Testing:

The script test_schema.py prints the schema text produced by get_schema(). The script test_prompt.py builds the complete prompt for the sample question “Show all diabetic patients older than 60” and prints it. These tests confirm that the schema is read correctly, that the question is inserted into the prompt and that all rules appear in the final prompt text.

5.14 Gemini Connection Testing:

The script test_gemini.py creates a Google GenAI client using the API key and asks the model to reply with a single word. A successful reply confirms that the API key is valid and that the

network connection to Gemini is working. The script test_models.py lists all the models that are available to the API key, which helps in selecting a suitable model name for the project.

5.15 SQL Generation Testing:

The script test_sql_generator.py calls generate_sql() with the question “Show all diabetic patients older than 60” and prints the SQL returned by Gemini. To evaluate the generated query, a reference query was run directly on the database:

```
SELECT * FROM patients
WHERE "Medical Condition" = 'Diabetes' AND Age > 60;
```

This reference query returns 3,419 records, so a correct generated query is expected to return the same number of rows. This approach of comparing the output of the generated query with a known result can be used to measure the accuracy of the text-to-SQL component.

5.16 Error Handling:

Error handling is implemented in the following ways:

- Database connections are closed in a finally block, so connections are not left open when a query fails.
- The interface does not call the AI model when the question box is empty.
- Markdown formatting in model output is removed before the SQL is used.

More complete handling of a missing API key, network failure, model errors and invalid SQL is included in the future scope.

5.17 Security Measures:

The project implements the following security measures:

- API key stored in an environment file instead of the source code
- Prompt rules that restrict generated queries to SELECT statements
- Database access through a dedicated module
- Connections closed after every query

A programmatic check that rejects any generated query that does not begin with SELECT is planned as an additional safeguard, since prompt rules alone cannot guarantee the behaviour of a language model.

RESULT, CONCLUSION AND FUTURE SCOPE

6.1 Result:

The Hospital AI Assistant successfully demonstrates how a generative AI model can be combined with a relational database. A healthcare dataset of 55,500 patient records was converted into an SQLite database, and a Streamlit interface was developed to view the records and to ask questions through Google Gemini. The schema reader, prompt builder and SQL generator were implemented and tested, and they are designed to convert natural-language questions into SQLite queries that follow the database schema and the safety rules.

The project demonstrates the practical integration of Python, Streamlit, SQLite, prompt engineering and Generative AI in a single application.

6.2 Conclusion:

The Hospital AI Assistant demonstrates the integration of generative AI with standard software engineering practices. The project combines a Streamlit frontend, an SQLite database, a modular set of Python components and Google Gemini into a unified application.

The system allows users to explore hospital data and to ask questions in natural language. By supplying the database schema and strict rules to the AI model, the project shows how text-toSQL can make structured data accessible to users who do not know SQL.

From a software engineering perspective, the project demonstrates the importance of modular architecture, separation of responsibilities, environment-based configuration and componentlevel testing.

Overall, the project provided practical exposure to modern AI application development and showed how generative AI can be combined with databases to build a useful real-world application.

6.3 Future Scope:

The project can be further enhanced by:

- Completing the orchestrator so that each question is routed to the SQL agent or the general AI agent.
- Executing the generated SQL from the interface and displaying the result table.
- Adding a validation layer that accepts only SELECT queries before execution.
- Summarizing query results in natural language so that answers are grounded in the stored records.
- Cleaning the dataset (duplicates, negative billing amounts, name formatting) and using column names without spaces.
- Adding user authentication, role-based access control and anonymization of sensitive patient information.
- Adding charts, an analytics dashboard and stored conversation history.
- Retrieval over medical reports and PDF documents using embeddings and a vector database, with voice and multilingual support.

-
-

Moving to a scalable database and deploying the application on the cloud.
Evaluating the accuracy of the generated SQL using a set of test questions with

known answers.

REFERENCES

1. Python Software Foundation, Python Documentation.
2. Streamlit, Streamlit Official Documentation.
3. SQLite, SQLite Official Documentation.
4. Pandas, Pandas Documentation.
5. Google AI, Gemini API and Google GenAI Documentation.
6. python-dotenv, python-dotenv Documentation.
7. Healthcare Dataset (healthcare_dataset.csv), publicly available healthcare dataset used for the project.

8. Submitted Project Archive, Hospital-AI-Assistant.zip.